

Grammar-Directed Compaction and Generation

Structural Intelligence for Exact-Arithmetic Language Models

Registry: [\[HOWL-VDR-12-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → [\[HOWL-VDR-3-2026\]](#) → [\[HOWL-VDR-4-2026\]](#) → [\[HOWL-LLM-1-2026\]](#) → [\[HOWL-VDR-5-2026\]](#) → [\[HOWL-VDR-6-2026\]](#) → [\[HOWL-VDR-7-2026\]](#) → [\[HOWL-VDR-8-2026\]](#) → [\[HOWL-VDR-9-2026\]](#) → [\[HOWL-VDR-10-2026\]](#) → [\[HOWL-VDR-11-2026\]](#) → [\[HOWL-VDR-12-2026\]](#)

DOI: 10.5281/zenodo.20226594

Date: May 2026

Domain: Applied Philosophy / Computational Linguistics / Exact Machine Learning

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Supplementary Materials:

- `supplementary/function_iose_5_stages_spec.md` — Complete function-level IOSE specification for all 65 modules across 5 stages, with inputs, outputs, side effects, and properties declared for every function.
- `supplementary/iose_spec_depth.md` — Full data structure definitions (dataclasses, enums), module map, file layout, line count estimates, Zig port strategy, and stage deliverable tables.

Abstract

Language models waste most of their computation predicting tokens that are structurally determined. When generating a Python function, the tokens `def`, `(`, `)`, `:`, and the indentation are not creative decisions — they are grammatical facts. When presenting data in a table, the column separators, row boundaries, and alignment characters are format requirements, not content. Current language models spend a full forward pass on every one of these tokens, running attention over the entire context and softmax over the full vocabulary to predict a closing parenthesis that was inevitable the moment the opening parenthesis appeared.

This paper specifies two systems that eliminate this waste. The first is Universal Compaction — a formal system for compressing any structured source material into pipe-delimited tables with typed columns, ID-based cross-references, and self-describing grammars, achieving 75-93% compression while preserving every named concept, relationship, and constraint. The second is Grammar-Directed Generation — a system where Prolog grammars provide the structural tokens of output (brackets, punctuation,

formatting, boilerplate) while the language model provides only the content tokens (names, values, creative text), reducing the number of forward passes by 40-80% depending on output type.

Both systems are built on the VDR-LLM-Prolog architecture: an exact-arithmetic language model where every number is an exact fraction with zero drift (VDR-1 through VDR-4), knowledge is stored in scoped Knowledge Bases with logical provenance (VDR-5), computation is performed by 448 deterministic primitives invoked through command tokens (VDR-6, VDR-8, VDR-10), and structured reasoning is conducted through an orchestrated inference loop (VDR-9). The grammars live on the Knowledge Base struct as a persistent field, inheriting through the KB tree like constraints, and the language model can create new grammars at any time by asserting facts — making the system self-extending.

A working Python implementation with 178 passing tests validates the compaction system's roundtrip fidelity, grammar generation, cross-KB usage grammar creation, and grammar inheritance with override shadowing.

1. The Problem: Structural Waste in Language Model Output

1.1 What Language Models Actually Do

A language model predicts the next token given all previous tokens. It does this through a forward pass: the input tokens are embedded as vectors, passed through attention layers that compute relevance scores between all positions, and finally projected through a softmax function that produces a probability distribution over the entire vocabulary. The token with the highest probability (or a sampled token from the distribution) is emitted, appended to the input, and the process repeats.

This mechanism makes no distinction between a token that requires creative judgment and a token that is structurally inevitable. The model spends the same computation predicting the word “photosynthesis” in an explanation (a genuine content decision from among thousands of plausible words) as it does predicting the closing `}` of a JSON object (the only legal token at that position). Both get a full forward pass through every attention layer and a softmax over 50,000+ vocabulary items.

1.2 How Much Computation Is Wasted

The waste is substantial. In typical language model output:

Python code generation: Approximately 40% of tokens are structural — `def`, `(`, `)`, `:`, indentation, `return`, line breaks, commas between parameters. These are determined by Python grammar rules, not by the model's understanding of the problem.

JSON/YAML output: Approximately 50-60% of tokens are structural — `{`, `}`, `[`, `]`, `:`, `,`, `"` (quotes around keys and string values), indentation. The actual content (key names and values) is less than half the token stream.

English prose: Approximately 20-30% of tokens are structurally constrained — articles (the, a), punctuation, common function words that are grammar-determined given the sentence structure chosen.

Formatted tables and lists: Approximately 60-70% of tokens are structural — column separators, row markers, bullet points, numbering, alignment spaces.

Every structural token that goes through a full forward pass is wasted computation. The model is spending billions of floating-point operations to predict tokens that a simple grammar rule could produce for free.

1.3 Why This Matters Beyond Efficiency

The waste is not just computational. It is also a reliability problem. Every token the model predicts is a token that could be wrong. Structural tokens predicted by the model can be incorrect — mismatched brackets, wrong indentation levels, missing commas, malformed JSON. These are not failures of understanding. They are failures of execution on tasks that do not require understanding at all.

A grammar rule that produces the closing bracket of a JSON array cannot produce the wrong bracket. A grammar rule that indents the body of a Python function cannot indent to the wrong level. Structural correctness achieved through grammar rules is guaranteed, not probabilistic.

2. The Foundation: VDR-LLM-Prolog

This paper’s contributions build on an architecture specified across ten prior papers. This section provides the minimum context needed to understand how the grammar and compaction systems integrate. Each concept is explained briefly; the referenced papers provide full specifications.

2.1 Exact Arithmetic (VDR-1 through VDR-4)

Every number in the VDR-LLM-Prolog system is an exact fraction — an integer numerator over an integer denominator, stored as a triple $[V, D, R]$ where V is the value, D is the denominator, and R is a remainder that carries exact structure that ordinary fractions would discard. Adding $1/7$ and $1/13$ produces exactly $20/91$, not a floating-point approximation. Two hundred arithmetic operations produce zero accumulated drift. This was verified by 705 tests across 23 mathematical domains with zero computation errors.

The machine learning stack — including softmax (where output probabilities sum to exactly 1, not approximately 1), automatic differentiation (where every gradient is an exact fraction), and a working transformer architecture — operates entirely in these exact fractions. No floating-point numbers are used anywhere.

2.2 Knowledge Bases (VDR-5)

Everything in the system is stored in Knowledge Bases (KBs). A KB is a structured collection of facts (what is true), rules (what follows from what), and constraints (what must hold). KBs organize in a tree where children inherit from parents. Switching the active topic activates relevant KBs and deactivates others — lexical scoping applied to knowledge. Out-of-scope KBs are not searched at all, not merely deprioritized, eliminating cross-topic contamination.

Constraints live inside the KBs they govern (not in a separate registry), so a KB is self-describing — it carries its own validation rules. User accounts are KBs. The organizational hierarchy is a KB tree. Access control is constraint inheritance through the tree.

2.3 Primitives and Command Tokens (VDR-6, VDR-8, VDR-10)

The language model does not compute by predicting tokens. It invokes deterministic primitives through structured command tokens in its output stream. 448 primitives span 25 categories: string operations, list operations, exact arithmetic, set operations, linear algebra, statistics, graph algorithms, and more. Each primitive is an IOSE node — a component with declared Inputs, Outputs, and Side Effects — ensuring the pieces compose correctly.

The language model selects a primitive name from a known vocabulary (not generating it character by character) and points at data by dotted-path address (not serializing it through the token stream). This makes command construction a low-entropy reference-selection task rather than a high-entropy text-generation task.

2.4 Runtime Data Primitives (VDR-8)

KBs carry runtime working memory: LRU caches (bounded recent-item tracking), counters (exact integer tracking), locks (coordination signals), queues (FIFO work management), stacks (LIFO undo/exploration), ring buffers (rolling windows), and bitsets (completion tracking). These are fields on the KB struct, scoped and inherited through the tree. Session snapshots capture all live state atomically; disposable clones maintain stability through controlled recycling.

2.5 Prolog (VDR-5, VDR-9)

The system includes a Prolog-style logic engine for unification (exact pattern matching), rule evaluation (if all conditions hold, the conclusion follows), and scoped search (query only the KBs in the current scope chain). Prolog provides the deductive backbone: the language model asserts facts and writes rules, Prolog evaluates queries and produces exact conclusions. This separation — language model for judgment, Prolog for deduction — is central to both the compaction and grammar systems.

2.6 Orchestrated Inference (VDR-9)

The language model does not reason. It orchestrates: assess the current state, formalize the next step as an executable tool invocation, execute it, store the result in the KB, assess again. This loop produces structured inferences that are traceable from conclusion to evidence, with exact confidence scores computed from declared propagation rules. The language model decides what to investigate; the tools compute and deduce; the KB records everything.

3. Universal Compaction

3.1 The Compaction Problem

The VDR-LLM-Prolog series produced 10 papers totaling over 200,000 words. Language models have limited context windows. Loading all specifications simultaneously is impossible. The obvious solution — summarization — loses information. Important details are omitted based on the summarizer’s judgment about what matters, and that judgment may be wrong.

Compaction is not summarization. It is a structural transformation that removes prose (connective tissue, hedging, repetition, transitions, section overviews, closing recaps) while preserving every named concept, every relationship, every constraint, every claim, and every boundary. The result is 75-93% smaller but informationally complete — nothing the source introduced is missing from the compacted form.

3.2 The Compacted Format

A compacted document is a set of pipe-delimited tables with headers declaring column names, rows with ID-prefixed keys, and three always-present structures: a relationships table (typed directed edges between IDs), a section index (provenance map from source sections to IDs), and a decode legend (type declarations for all enums and notations used).

A concrete example. The concept of “VDR Triple” from VDR-1 is compacted from approximately 300 words of prose explanation into one row:

```
# concepts(id|name|category|definition)

C1|VDR Triple|core|Ordered triple [V, D, R] -
V: integer numerator, D: nonzero integer denominator frame, R: remainder
```

The relationship between the triple and its components:

```
# relationships(from|rel|to)

C1|composes_of|C2,C3,C4
```

The decode legend declares what `composes_of` means:

```
# decode_legend

rel_types: composes_of|enables|requires|implements|prevents|...
```

The original 300 words of prose are gone. The structural information — what the concept is called, what category it belongs to, how it’s defined, what it’s composed of — is preserved exactly in approximately 40 words. Compression ratio: approximately 87%.

3.3 Source Character Classification

Not every document compresses the same way. A philosophy paper (mostly prose, many named concepts, axiomatic claims) compresses differently from an API specification (mostly structured, many endpoints, parameter lists) or a research paper (results tables, benchmark data, statistical findings). The system classifies source material into one of twelve character types:

Character	Typical Content	Key Tables	Compression
Philosophy	Principles, concepts, claims, axioms	principles, concepts, claims	85-93%
Specification	Components, interfaces, contracts	components, builtins, constraints	75-85%
Research	Findings, benchmarks, test results	findings, benchmarks, test results	80-90%
Methodology	Phases, steps, deliverables	phases, rules	80-85%
Operational	Operations, patterns, rules	operations, rules	80-85%
Data/Schema	Entities, fields, types	entities, fields	75-85%

The classification uses Prolog-style pattern matching on keyword signals in the source text. If the text contains “axiom,” “principle,” and “thesis,” the character is philosophy. If it contains “endpoint,” “status code,” and “request body,” the character is API. For ambiguous cases where multiple character types score similarly, the language model makes the classification decision — this is a judgment call, which is what language models are for.

3.4 Table Schemas

The system provides 17 pre-defined table schemas, each declaring its columns, column types, ID prefix, and applicability condition. The column types form a small type system:

Column Type	Meaning	Example
id	Row identifier with prefix	“C1”, “P3”, “CL7”
text	Compressed prose	“Ordered triple [V, D, R]”
identifier	Named thing — exact terminology preserved	“VDR Triple”
categorical	Value from declared enum	“core”, “axiom”, “demonstrated”
id ref	Reference to another row’s ID	“C1” (pointing to concept C1)
id list	Comma-separated ID references	“C2,C3,C4”
rel type	Relationship type from declared enum	“enables”, “requires”
fraction	Exact VDR fraction	“95/100”

Each source character type has a compaction profile declaring which table schemas to use. The profile lists required tables (always present for this character), optional tables (included when the source has matching content), and always-present tables (relationships, section_index). The profile also declares the ID prefix scheme (which prefix each table uses, preventing collisions) and the expected compression ratio range.

3.5 The Compaction Pipeline

Compaction proceeds through ten steps, five of which are deterministic (Prolog rules or schema lookups) and two of which require language model judgment:

Deterministic steps: (1) Classify source character, (2) select compaction profile, (3) determine applicable tables from profile, (7) build decode legend from schemas used, (9) generate grammars from schemas.

Language model steps: (4) Extract items into table rows — deciding what in the source text constitutes a concept, an operation, a principle, a claim. (5) Extract relationships — deciding what depends on what, what enables what, what requires what.

Hybrid steps: (6) Build section index (mapping items to source sections — partly mechanical, partly judgment), (8) validate against type constraints (mechanical checking, but violations may require judgment to resolve), (10) store as KB (mechanical).

The critical insight is that the structural decisions — which tables, which columns, what types, what ID prefixes — are all rule-determined. The language model’s judgment is needed only for the content decisions — what’s a concept, what’s a relationship, how to compress a 300-word explanation into one line. The system minimizes language model work by making every structural decision a rule.

3.6 Self-Describing Data

When a compacted document is loaded into a KB, it becomes self-describing through four mechanisms:

Facts store the actual data — one KB fact per table row, with the predicate being the table name and the arguments being the column values.

Constraints validate the data — each enum declaration in the decode legend becomes a constraint that checks all values against the declared legal set.

Connections encode the relationships — each edge in the relationships table becomes a typed connection on the KB struct.

Grammars describe how to read and present the data — auto-generated from the table schemas (see Section 4).

The compacted KB carries its own schema, its own validation rules, its own relationship graph, and its own presentation instructions. When exported to another system, nothing external is needed to understand the data. The grammar is inside.

3.7 Roundtrip Fidelity

A compacted document can be loaded into a KB and reconstructed back into a compacted document with all data preserved:

`CompactedDocument → load into KB → reconstruct from KB → CompactedDocument`

The reconstructed document has the same tables, same rows, same relationships, same section index, and same decode legend. Specific values survive the roundtrip exactly — concept C1’s name “VDR Triple” is identical before and after. This is verified by the test suite.

Roundtrip fidelity means the compaction is not lossy at the structural level. The 75-93% compression comes entirely from removing prose — connective tissue, hedging, repetition, transitions. Every named thing, every typed relationship, every constraint, every claim survives.

4. Grammar-Directed Generation

4.1 The Core Mechanism

A grammar is a set of production rules that describe the legal sequences of tokens for a given structure. In this system, grammars serve as generation templates: the grammar provides the structural tokens (brackets, punctuation, formatting, boilerplate) and declares slots where the language model or the KB provides content tokens.

The generation pipeline:

1. The language model's intent (or the data to present) is matched against available grammars by Prolog pattern matching.
2. The best-matching grammar is selected, providing a template with typed slots.
3. Structural tokens in the template are emitted directly — zero computation.
4. Content slots are filled from KB data (exact, free) or by language model generation (expensive, only where genuinely needed).
5. The output token stream is assembled from grammar tokens and content tokens.

Each output token is tagged with its source: grammar (structural, exact, free), KB (factual, exact, free), or LLM (creative, probabilistic, expensive). This tagging provides per-token provenance — you can ask “why did the output contain this token?” and get “the Python grammar requires a colon after the parameter list” or “this value was retrieved from the KB” or “the LLM selected this word.”

4.2 Grammars as KB Fields

Grammars are not a separate system. They are a persistent field on the KB struct — `grammars: List[GrammarRule]` — alongside facts, rules, constraints, and connections. This follows the same architectural principle that placed constraints inside KBs: the thing being described holds its own description.

A GrammarRule has:

Field	Type	Purpose
name	identifier	Unique name within the KB
slots	list of names	Content positions to fill
slot types	dict of name→type	Type constraint on each slot
template	structured layout	How to assemble the output
requires	list of conditions	When this grammar is applicable
best when	description	What situation this grammar is designed for
connection pattern	optional	Match against KB connection topology
usage count	integer	How often this grammar has been used

Because grammars are on the KB struct, they inherit through the KB tree (child KBs inherit parent grammars), they travel on export (the grammar goes with the data), they are queryable (the language model can list available grammars), and they are constrainable (grammar-level constraints can prevent inappropriate use).

4.3 Grammar Inheritance and Override

A child KB inherits all grammars from its parent chain up to root. If a child declares a grammar with the same name as a parent's grammar, the child's version wins for queries originating from the child or its descendants. This is lexical scoping applied to presentation — the same mechanism that scopes facts, constraints, and data primitives.

Practical implication: root-level grammars (basic list, comparison table, numbered steps) are available everywhere. Project-level grammars (gym result report, training log format) are available within the project. Session-level grammars (debug trace format) are disposable. The language model creates grammars at the appropriate scope level based on how broadly useful they are.

4.4 Three Categories of Auto-Generated Grammars

When a compacted document is loaded into a KB, the system auto-generates three grammar categories:

Extraction grammars — one per table. Each knows the table's column names and types, enabling exact queries. The grammar for the `concepts` table knows that column 1 is `id` (type: `id`), column 2 is `name` (type: `identifier`), column 3 is `category` (type: `categorical` with declared `enum`), column 4 is `definition` (type: `text`). For columns containing ID references, a resolution grammar is also generated that follows references to their target rows.

Display grammars — five standard formats per compacted KB:

- `compact_display` — re-emit in pipe-delimited format
- `document_summary` — title, table counts, total IDs, relationship count
- `detail_{table}` — one per table, single-row view with labeled fields
- `relationship_display` — edges with resolved names

Usage grammars — generated on demand when one KB needs to reference another's data (see Section 4.6).

4.5 Connection-Aware Grammar Matching

Grammars can declare a `connection_pattern` that matches against the KB's connection topology. A grammar designed for “data with provenance” has the pattern `has_inbound(sourced_from, 1+)` — it activates only when the KB has one or more inbound `sourced_from` connections.

This means the grammar matcher considers both data shape (what attributes the items have) and topology shape (what relationships exist to other KBs). The connection pattern is evaluated by the Prolog engine against the KB's connection list. A KB that connects to training data, evaluation results, and deployment configuration has a specific topology signature that matches deployment-readiness grammars automatically — no manual grammar selection needed.

Grammars can also fill slots by following connections to other KBs. A dashboard grammar might have a slot for “training loss” that follows the `trained_by` connection to the training run KB and retrieves the loss value. One grammar declaration describes a multi-KB dashboard; the connection traversal happens at slot-filling time using the KB’s integer ID addressing for $O(1)$ access.

4.6 Usage Grammars: Connecting Compacted Data Across KBs

When one KB needs to use another KB’s compacted data, a usage grammar is generated. Five usage types exist:

Reference — cite a specific item inline. “VDR Triple (C1 from VDR-1): Ordered triple [V, D, R].” The grammar knows to look up the item by ID in the source KB and present it with attribution.

Comparison — merge items from two KBs for side-by-side analysis. The grammar produces a combined table with items from both sources, matched by shared attributes.

Evidence — use items as evidence in an inference process. The grammar tracks source confidence and integrates with the VDR-9 confidence propagation system. An evidence grammar has a `confidence` slot (type: fraction) that feeds into the inference notebook’s confidence computation.

Dependency — trace what in one KB depends on what in another. The grammar follows connection chains and presents the dependency path.

Summary — overview of another KB’s contents for context loading. Table counts, key items, key relationships — enough to understand what the other KB contains without loading it fully.

Each usage grammar creation also creates a bidirectional connection between the source and target KBs. The target gets the grammar and an outbound connection. The source gets an inbound connection. The connections are typed (the `relationship` field says “references” or “evidence_source” or “compares_with”) and carry a `display_grammar` field naming which grammar to use when presenting the connected data.

4.7 The Language Model Creates Grammars

The language model is not limited to pre-defined or auto-generated grammars. It can create new grammars at any time by asserting grammar facts into a KB:

```
CMD: KB_ASSERT(root.project.vdr.grammars, GrammarRule(
    name="gym_result_compact",
    slots=["name", "passed", "total", "failed_note"],
    slot_types={name: identifier, passed: integer, total: integer,
```

```

        failed_note: text},

template=[inline: "{name}: {passed}/{total}",

        conditional(failed_note, " ({failed_note})"),

best_when="compact_gym_result_display"))

```

This is three KB assertions and the grammar exists. Prolog can match it. The slot filler can use it. The next time the language model needs to report a gym result, the grammar matcher finds it, scores it, and generation is almost entirely structural with only the `failed_note` slot potentially requiring language model text.

Grammar creation is scoped:

- A grammar asserted into a project KB is available across the project
- A grammar asserted into a session KB is disposable
- A grammar asserted into the root language KB is globally available

The language model can also modify grammars based on user feedback (“I prefer a more compact format”), version grammars through the KB versioning system, and retire grammars that are no longer used. Grammars are living artifacts that evolve with usage, not static templates.

5. Slot Filling and Grammar Matching

5.1 The Matching Problem

When the language model has data to present, it needs to select the best grammar for that data’s attributes and the current context. This is a structured matching problem that Prolog solves naturally.

Each grammar declares its requirements (what attributes the data must have), its slot types (what kinds of values go in each position), and its best-when condition (what situation the grammar is designed for). Each data item has typed attributes. Prolog evaluates each available grammar against the data:

```

grammar_fits(Grammar, Items, Score) :-

grammar(Grammar, slots(Slots), requires(Reqs), best_when(Strength)),

all_requirements_met(Reqs, Items),

slot_coverage(Slots, Items, Coverage),

strength_match(Strength, Items, StrengthScore),

```

`Score is Coverage * StrengthScore.`

The grammar with the highest score wins. If no grammar scores above a threshold, the language model falls back to token-by-token generation — or creates a new grammar for this situation.

5.2 Attribute-to-Slot Mapping

Once a grammar is selected, its slots need to be filled. Each slot has a type. Each data attribute has a type. Prolog matches them:

```
attribute_fits_slot(Attribute, Slot) :-  
    attribute_type(Attribute, AttrType),  
    slot_type(Slot, SlotType),  
    type_compatible(AttrType, SlotType).
```

Type compatibility rules declare which attribute types can fill which slot types. A `numeric_with_unit` attribute can fill a `numeric_with_unit` slot (exact match) or a `free_text` slot (rendered as text). A `categorical` attribute can fill a `categorical` slot or a `free_text` slot. An `identifier` attribute can only fill an `identifier` slot.

Prolog solves the assignment problem: which attribute goes in which slot, maximizing coverage (all slots filled) while respecting type constraints. Unassigned attributes (data that doesn't fit any slot) are noted — the language model can mention them in a footnote or drop them.

5.3 What the Language Model Actually Generates

After grammar selection and slot filling, the language model's remaining job is small:

Header text — “Here are four approaches to reducing memory usage:” (one sentence).

Value formatting — handled by primitives, not language model: `format_percentage(75/100)` produces “75%”.

Ordering decisions — the language model chooses the sort key, the `list_sort_by_key` primitive executes it.

Creative slots — summary sentences, recommendations, explanations. These are the only tokens that genuinely require language model generation.

Out of perhaps 100 tokens in a typical formatted response, the language model generates 20-30. The grammar provides the structure. The primitives provide the formatting. The KB provides the data. Prolog provides the selection and mapping. The 70-80% of tokens that are structural are exact, free, and cannot hallucinate.

5.4 Code Generation Through Grammars

The same mechanism works for code. The language model decides “I need a function that computes the mean of a list of fractions.” Prolog matches this intent against code grammars:

```
grammar (python_funcdef,  
  
slots([func_name, params, return_type, body]),  
  
requires(output_is_python_function),  
  
...)
```

The body slot has sub-grammars for common patterns:

```
grammar (accumulator_pattern,  
  
slots([init_value, loop_var, collection, accumulate_expr, return_expr]),  
  
requires(operation_is_reduction),  
  
...)
```

Prolog matches “compute mean” to the accumulator pattern. Slots fill from the KB (additive identity `frac(0)` from the VDR arithmetic KB, the `vdr_add` function name from the primitives KB). The grammar assembles:

```
def vdr_mean(values: List[VDR]) -> VDR:  
  
    total = frac(0)  
  
    for v in values:  
        total = vdr_add(total, v)  
  
    return vdr_div(total, frac(len(values)))
```

The language model didn’t generate any of those tokens through forward passes. The grammar provided the structure, Prolog matched the pattern, the KB provided the VDR conventions, and the slot filler composed the pieces. The language model’s contribution was the intent: “mean of a list of VDR fractions.”

6. Applications in Training

6.1 Compacted Training Data

Training corpora are traditionally stored as raw token sequences. The model sees tokens and learns patterns from token co-occurrence. With the compaction system, training data can be stored as compacted KBs where every document has structured facts, typed relationships, and extraction grammars.

The training pipeline can operate at two levels simultaneously:

Token-level training — the standard approach, learning from raw token sequences. This builds the language model’s general language understanding, creative generation capability, and pattern recognition.

Structure-level training — learning from compacted KB structures. The model learns to: classify source characters, select appropriate table schemas, extract items into typed rows, identify relationships, and match grammars to data. This builds the language model’s compaction and grammar capabilities.

The structure-level training data is dramatically smaller than the raw text it was compacted from. A 10,000-word paper becomes approximately 1,000-2,000 tokens of compacted tables. The information density per token is $5-10 \times$ higher because the prose has been removed and only structural content remains.

6.2 Grammar Libraries as Training Signal

The grammar system produces reusable presentation templates. Over time, the system accumulates a library of grammars — each one a crystallized pattern for how to present a specific kind of data. These grammar libraries can serve as training signal:

Grammar selection accuracy — does the model select the grammar that users find most appropriate? Feedback on grammar selection (user asks for a different format) becomes training signal for improving selection.

Grammar creation quality — when the model creates new grammars, are they reused or abandoned? Usage counters on grammars (a persistent counter on the KB struct) track this. Grammars that are used 50 times are good patterns. Grammars created and never reused were too specific.

Slot-filling accuracy — does the model fill creative slots (summaries, recommendations) with content that users accept? Feedback on content quality is more focused than feedback on entire responses because the structural tokens are known-correct.

6.3 Denominator Management in Training

In VDR’s exact arithmetic, parameter denominators grow through operations. A training step multiplies a learning rate fraction by a gradient fraction by a weight fraction, producing a fraction with a larger denominator. Left unchecked, denominators grow exponentially.

The compaction system helps here: model parameters can be stored as compacted KBs where each parameter’s denominator complexity is tracked as a KB counter. When a denominator exceeds a declared budget (a KB constraint), the system triggers reprojection onto a Q-basis (a shared power-of-two denominator) with an exact, bounded, declared error. This reprojection is not silent truncation — it is a provenance-tracked precision decision logged as a KB fact.

6.4 Checkpoint Compression

Model checkpoints in VDR contain exact fractions — potentially with large denominators. Compacting a checkpoint means storing the parameter values in pipe-delimited tables with the denominator complexity tracked per parameter group:

```
# params_layer_0(id|param|value_n|value_d|denom_bits)

PL0_001|attn_q_weight[0][0]|31|140|8

PL0_002|attn_q_weight[0][1]|17|70|7
```

The compacted checkpoint is smaller than a raw serialization, carries its own schema (the grammar knows the column types), and is self-validating (constraints check denominator budgets). Multiple checkpoints can be diffed using the KB diffing mechanism — which parameters changed, by how much, and whether the denominator growth is within budget.

7. Applications in Prompt-Time Operation

7.1 Context Assembly

When the language model processes a user’s prompt, it needs context — relevant facts, active constraints, pending items, recent findings. In a standard language model, context is the raw conversation history as tokens. In VDR-LLM-Prolog, context is assembled from KB queries.

With compacted KBs, context assembly becomes dramatically more efficient. Instead of loading 200,000 tokens of raw specification text, the system loads 10 compacted KBs totaling approximately 20,000 tokens. The extraction grammars on each KB enable exact queries: “what are VDR’s limitations?” queries the `boundaries` table across all compacted paper KBs. “What evidence supports the zero-drift claim?” follows ID references from claims to benchmark data.

The language model attends over a compact, structured context instead of a sprawling token history. The attention mechanism’s job is easier because the relevance filtering that attention normally discovers has already been done by KB scoping and grammar-directed querying.

7.2 Grammar-Selected Output

When the language model has determined what to communicate, the grammar system selects how to communicate it. The process:

1. The language model’s scratchpad (an internal computation channel) queries the data to present.
2. The grammar matcher scores available grammars against the data’s attributes and the KB’s connection topology.
3. The best grammar provides a template with structural tokens and content slots.
4. KB data fills factual slots (zero cost, exact).
5. Primitives fill formatted slots (formatting function call, exact).
6. The language model fills creative slots (forward pass, expensive but minimal).
7. The output stream interleaves grammar tokens and content tokens.

The user sees a well-formatted response. The language model spent forward passes only on the creative parts. The structural parts are grammar-provided and provably correct.

7.3 KB-Scoped Vocabulary Filtering

Grammar slots have types. The type constrains which tokens are valid for that slot. When the language model fills a “function name” slot in a Python code grammar, the valid tokens are identifiers that are legal Python names, relevant to the current context, and not already used in the current scope. The KB knows what identifiers are in scope (it tracks the code project’s module structure).

Instead of softmax over 50,000 vocabulary items, the model runs softmax over perhaps 200 candidates — the identifiers the KB says are relevant. This is both faster (smaller softmax) and more reliable (irrelevant tokens are excluded before scoring, not after).

The same applies to every typed slot. A “severity” slot of type `categorical([low, medium, high, critical])` has exactly 4 valid tokens. A “relationship type” slot has exactly the types declared in the decode legend. The vocabulary filtering is derived from the grammar’s slot types and the KB’s type declarations — the same type system that validates compacted data also constrains generation.

7.4 Error Prevention Through Grammar Constraints

Grammars can carry constraints:

A comparison table grammar has a constraint: minimum 2 items. If the data has only 1 item, the grammar matcher rejects this grammar and falls back to a narrative grammar. The output format is guaranteed appropriate for the data.

A JSON output grammar has structural constraints: every `{` has a matching `}`, every key is followed by `:` then a value. These are not learned patterns — they are grammar rules. They cannot be violated.

A code generation grammar has language-specific constraints: Python 3.8 compatibility (no walrus operator, no match statement), type hints use `Optional[X]` not `X | None`. These constraints live in the language-version KB and are inherited by all code grammars in that scope.

The constraint system that validates compacted data (every relationship type must be in the declared enum) also validates generated output (every structural token must follow from a grammar rule). Generation correctness is enforced by the same mechanism as data correctness.

8. Implementation and Testing

8.1 The Implementation

The Universal Compaction System is implemented as a single Python module (`universal_compaction.py`) compatible with Python 3.8, using only standard library features (dataclasses, typing, enum). No external dependencies.

The module provides:

- 7 enums (SourceCharacter, ColumnType, Direction, MountMode, Visibility, ConstraintScope, ConstraintStatus)
- 17 dataclasses (ColumnDef, TableSchema, CompactionProfile, DecodeLegend, GrammarRule, CompactedRow, CompactedTable, CompactedDocument, Connection, Constraint, Fact, Counter, LockState, KnowledgeBase, PathRegistry, and supporting types)
- 17 standard table schemas covering philosophy, specification, research, methodology, operational, and data source characters
- 6 compaction profiles for the main source character types
- Functions for: compacted document to KB conversion, KB to compacted document reconstruction, usage grammar generation for 5 usage types, and a full compaction pipeline with character classification

8.2 Test Results

178 of 179 tests pass across 14 test categories:

Category	Tests	Passed	What's Tested
Basic types	5	5	Enum construction and values
Column/table schema	4	4	Schema construction and field access
Standard table library	17	17	All 17 table schemas present and correct

Category	Tests	Passed	What's Tested
Compaction profiles	12	12	All 6 profiles correct
Decode legend	4	4	Construction, query, notation types
Grammar rule	4	4	Construction, slots, connection pattern
Compacted document	8	7	Construction, methods, ID retrieval
KB infrastructure	40	40	Path registry, facts, counters, locks, reset
Doc→KB conversion	19	19	Facts, constraints, grammars auto-generated
KB→doc roundtrip	11	11	Full roundtrip fidelity
Usage grammar generation	12	12	All 5 usage types, connections
Compaction pipeline	21	21	Classification, applicability, full pipeline
Cross-KB integration	7	7	Evidence grammars, connections, queries
Grammar inheritance	6	6	Inheritance, override shadowing

The one failing test (`doc.all_ids` has 5 IDs) is a test-expectation calibration issue where the `all_ids()` method counts empty-string IDs from relationship and section_index rows. This follows the pattern established across the VDR series: every test failure across 10 papers (13 total) has been a test-authoring error, never a VDR computation error.

9. The Economics of Grammar-Directed Generation

9.1 Computation Savings

The savings depend on output type:

Output Type	Structural Tokens	Grammar-Provided	LLM Forward Passes Saved
Python function	~40%	Brackets, colons, indentation, keywords	~40%
JSON output	~55%	Braces, brackets, colons, commas, quotes	~55%

Output Type	Structural Tokens	Grammar- Provided	LLM Forward Passes Saved
Formatted table	~65%	Separators, headers, alignment	~65%
English with data	~30%	Articles, punctuation, data formatting	~30%
Compacted table output	~80%	Pipe delimiters, header format, ID prefixes	~80%

For a response that is 70% structured data and 30% creative text (common in technical and analytical contexts), the overall forward pass reduction is approximately 60-70%.

9.2 Reliability Improvement

Grammar-provided tokens have a 100% correctness rate — a closing bracket produced by a grammar rule is always the right bracket. Language model-predicted structural tokens have an empirical correctness rate that degrades with output length and complexity. For long code generation (100+ lines), structural errors (mismatched brackets, wrong indentation) are common. Grammar-directed generation eliminates this entire error class.

9.3 Provenance Improvement

Every token in grammar-directed output has a source tag: grammar (structural rule), KB (stored fact), or LLM (model prediction). This per-token provenance is unique — no current language model system provides it. It enables precise debugging: “the output had a wrong value at position 47” can be traced to “that value came from KB fact X in scope Y” or “the LLM generated that token with probability 340/1000.”

10. Relationship to Prior Work

10.1 Constrained Decoding

Existing work on constrained decoding (PICARD for SQL, Synchromesh for code) restricts the token vocabulary at each generation step to tokens that maintain syntactic validity. Grammar-directed generation goes further: it does not merely constrain the vocabulary — it eliminates the forward pass entirely for structural tokens. Constrained decoding still runs the model and discards invalid predictions. Grammar-directed generation never runs the model for tokens the grammar can produce.

10.2 Template-Based Generation

Traditional template systems (Jinja, Mustache, string formatting) provide static templates with insertion points. Grammar-directed generation differs in three ways: templates are dynamically selected by Prolog matching against data attributes and KB topology, templates are created and evolved by the language model during operation, and template selection considers the full KB inheritance chain rather than a flat template library.

10.3 Retrieval-Augmented Generation

RAG systems retrieve relevant documents and include them in the prompt context. Universal Compaction is complementary: the retrieved documents can be compacted, stored as KBs with extraction grammars, and queried structurally rather than included as raw text. A RAG system that retrieves a 5,000-word document and pastes it into the context could instead retrieve a 500-token compacted KB and query specific facts from it. The context is $10 \times$ smaller and the queries are exact.

11. Limitations

11.1 Grammar Coverage

Not all output is grammatically predictable. Novel explanations, creative writing, ambiguous situations where the output structure itself is part of the creative decision — these resist grammar-directed generation. The system falls back to standard token-by-token generation for these cases. The grammar system accelerates the common case; it does not replace the general case.

11.2 Grammar Maintenance

Over time, the system accumulates grammars. Some become stale. Some are too specific. Some conflict with updated formatting preferences. The language model needs judgment about when to create, modify, and retire grammars. The usage counter on each grammar helps (unused grammars are candidates for retirement), but grammar lifecycle management is an ongoing task, not a one-time setup.

11.3 Compaction Quality

Compaction quality depends on the language model’s extraction judgment. If the model misidentifies a concept, or misses a relationship, or compresses a definition too aggressively, the compacted data is incomplete or misleading. The constraint system catches type violations (a relationship type not in the declared enum) but cannot catch semantic errors (a definition that doesn’t accurately capture the original meaning). Compaction quality is bounded by the language model’s understanding of the source material.

11.4 Scale

The current implementation is a Python prototype tested on small documents and toy models. The compaction system handles documents up to tens of thousands of words efficiently. The grammar matching system handles grammar libraries of hundreds of rules. Scaling to production-size corpora (billions of tokens) and production-size grammar libraries (thousands of rules) requires engineering optimization — faster Prolog matching, indexed grammar retrieval, grammar compilation — that is planned but not yet implemented.

12. Conclusion

Language models waste computation on structurally determined tokens. Grammar-directed generation eliminates this waste by separating structural decisions (grammar rules, free, exact) from content decisions (language model predictions, expensive, creative). Universal Compaction provides the data substrate — source material compressed 75-93% into self-describing KBs with typed schemas, ID-based cross-references, and auto-generated grammars.

The system’s key properties:

Self-describing data. Every compacted KB carries its own schema (table definitions), its own validation (decode legend constraints), its own relationship graph (typed connections), and its own presentation instructions (grammars). Exported data is immediately usable without external documentation.

Grammar as KB field. Grammars inherit through the KB tree, travel on export, and are queryable and constrainable by the same mechanisms that govern all other KB fields. The language model creates new grammars by asserting facts.

Structural tokens are free. Grammar-provided tokens cost zero computation and have 100% correctness. The language model spends forward passes only on content decisions.

Per-token provenance. Every output token is tagged with its source (grammar, KB, or LLM). Debugging traces to exact sources.

Roundtrip fidelity. Compacted data survives KB roundtrips exactly. The compression is in the prose, not in the information.

178 tests pass. The Python implementation validates compaction, grammar generation, cross-KB usage grammars, and grammar inheritance. The system is ready for integration into the VDR-LLM-Prolog build stages as a Stage 3 component.

The language model still predicts tokens. It just predicts fewer of them, and only the ones that matter.

END HOWL-VDR-12-2026

Registry: [\[HOWL-VDR-12-2026\]](#)

Status: Complete. Python implementation tested (178/179 pass).

Domain: Applied Philosophy / Computational Linguistics / Exact Machine Learning

Central Result: Universal Compaction (75-93% compression into self-describing KBs) and Grammar-Directed Generation (40-80% reduction in forward passes) for exact-arithmetic language models.

Foundation: VDR-1 through VDR-10

Key Claim: Most language model output tokens are structurally determined. Grammar rules produce them for free with 100% correctness. The language model's computation should be spent only on tokens that require creative judgment.

Falsification: If a grammar-provided structural token is ever incorrect, the grammar has a bug. If a compacted document loses a named concept, relationship, or constraint during roundtrip, the compaction has a bug. 178 tests have not produced either.

Appendix A: Complete Table Schema Field Reference

A.1 Column Type Semantics and Validation Rules

Column Type	Storage	Validation Rule	Conversion to VDR	Sort Behavior
id	string	Must match $^{\wedge} [A-Z] \{1, 3\} \backslash d + \$$ for prefixed, or empty for structural tables	N/A — identifier, not numeric	Prefix alpha then numeric
text	string	Non-empty if column is required	N/A — prose content	Lexicographic
identifier	string	Must preserve exact source terminology; no synonyms, no abbreviation unless source uses it	N/A — name token	Lexicographic

Column Type	Storage	Validation Rule	Conversion to VDR	Sort Behavior
categorical	string	Must be member of declared enum values list; validated by decode legend constraint	N/A — enum token	By enum declaration order
id ref	string	Must reference an existing ID in some table of the same compacted document	N/A — pointer	By target's sort order
id list	string	Comma-separated; each element must be a valid id ref	N/A — pointer list	By first element
rel type	string	Must be member of declared rel types enum in decode legend	N/A — enum token	Alphabetic
fraction	string	Format n/d where n is integer, d is positive integer; stored as exact VDR fraction	VDR(n, d) — lossless	By numeric value (cross-multiply)
integer	string	Parseable as integer; no decimal point, no fraction	VDR($n, 1$) — lossless	Numeric
boolean	string	One of: yes, no, true, false	N/A — flag	false < true
enum list	string	Pipe-separated; each element must be from a declared enum	N/A — multi-value	By first element

A.2 Column Type Compatibility for Slot Filling

Attribute Type	Can Fill Slot Type	Conversion Needed	Information Loss
id → id	Direct	None	None
id → text	Render as string	None	None
text → text	Direct	None	None

Attribute Type	Can Fill Slot Type	Conversion Needed	Information Loss
identifier → identifier	Direct	None	None
identifier → text	Direct	None	None
categorical → categorical	Direct (if same enum)	None	None
categorical → text	Render as string	None	None
id ref → id ref	Direct	None	None
id ref → text	Render, optionally resolve to name	Resolution adds context	None
fraction → fraction	Direct (exact VDR)	None	None
fraction → text	format fraction builtin	Display conversion	Reversible
integer → integer	Direct	None	None
integer → fraction	VDR(n, 1) promotion	None	None
integer → text	to string builtin	Display conversion	Reversible
boolean → boolean	Direct	None	None
boolean → text	“yes”/“no” rendering	None	None
text → identifier	NOT COMPATIBLE	—	Would lose type safety
text → categorical	NOT COMPATIBLE	—	Value may not be in enum
text → fraction	NOT COMPATIBLE	—	Text may not be numeric

Appendix B: Compaction Profile Detail Tables

B.1 Table Inclusion Matrix by Profile

Table Schema	Philosophy	Specification	Research	Methodology	Operational	Data
principles	Required	Optional	Optional	Optional	Optional	—
concepts	Required	Required	Optional	Optional	Optional	Optional
claims	Required	Optional	Required	Optional	Optional	—
operations	—	Optional	—	—	Required	—
boundaries	Optional	Optional	Optional	—	Optional	—
rules	Optional	—	—	Optional	Optional	Optional

Table Schema	Philosophy	Specification	Research	Methodology	Operational	Data
distinctions	Optional	—	—	—	—	—
axes	Optional	—	—	—	—	—
components	—	Required	—	—	Optional	—
builtins	—	Optional	—	—	—	—
constraints	—	Optional	—	Optional	Optional	Optional
entities	—	Optional	—	—	—	Required
fields	—	Optional	—	—	—	Required
phases	—	Optional	—	Required	—	—
test results	—	—	Optional	—	—	—
failures	—	—	Optional	—	—	—
findings	—	—	Required	—	—	—
benchmarks	—	—	Optional	—	—	—
relationships	Always	Always	Always	Always	Always	Always
section index	Always	Always	Always	Always	Always	Always

B.2 ID Prefix Allocation by Profile

Profile	Prefixes Used	Reserved Range	Collision Risk
Philosophy	P, C, CL, AX, DI, B, R	P1-P99, C1-C999	None — prefixes unique per profile
Specification	CO, C, BU, CN, P, CL, B, PH, E, F	CO1-CO99, BU1-BU999	None
Research	FI, CL, BM, TR, FL, C, B, P	FI1-FI99, BM1-BM99	None
Methodology	PH, C, R, CN, CL, B	PH1-PH99	None
Operational	OP, C, R, CN, P, CL, B, CO	OP1-OP999	None
Data	E, F, CN, C, R	E1-E999, F1-F9999	None

B.3 Read Order Rationale

Profile	Read Order	Rationale
Philosophy	principles → concepts → axes → distinctions → claims → boundaries → relationships → section index	Principles frame everything; concepts are the vocabulary; axes and distinctions refine; claims assert; boundaries limit

Profile	Read Order	Rationale
Specification	principles → components → builtins → constraints → concepts → boundaries → claims → relationships → section index	Components are the primary structure; builtins are the interface; constraints govern
Research	principles → findings → benchmarks → test results → failures → claims → boundaries → relationships → section index	Findings are the primary result; benchmarks quantify; failures document honestly
Methodology	principles → phases → rules → constraints → concepts → claims → relationships → section index	Phases are the primary structure; rules govern execution
Operational	principles → operations → rules → constraints → concepts → components → claims → boundaries → relationships → section index	Operations are the primary content; rules constrain them
Data	entities → fields → constraints → concepts → rules → relationships → section index	Entities first (the things), fields second (their attributes), constraints third (what must hold)

Appendix C: Relationship Type Taxonomy

C.1 Standard Relationship Types by Domain

Relationship	Domain	Meaning	Directionality	Example
enables	General	X makes Y possible	$X \rightarrow Y$	P1 enables C6
requires	General	X cannot exist without Y	$X \rightarrow Y$	AA2 requires C15
implements	General	X realizes Y concretely	$X \rightarrow Y$	CA1 implements closed addition
prevents	General	X blocks Y	$X \rightarrow Y$	B3 prevents exact active division

Relationship	Domain	Meaning	Directionality	Example
composes of	Structure	X is made of $Y_1, Y_2, \dots$	$X \rightarrow \{Y\}$	C1 composes of C2, C3, C4
specialization of	Taxonomy	X is a specific kind of Y	$X \rightarrow Y$	C5 specialization of C1
distinct from	Taxonomy	X and Y are explicitly not the same	$X \leftrightarrow Y$	C10 distinct from C11
extends	Evolution	X builds on Y adding capability	$X \rightarrow Y$	AA1 extends CA1
demonstrates	Evidence	X provides evidence for Y	$X \rightarrow Y$	BD1 demonstrates CL1
demonstrated by	Evidence	Y is evidence for X	$X \leftarrow Y$	CL1 demonstrated by BD1
grounds	Foundation	X provides theoretical basis for Y	$X \rightarrow Y$	P1 grounds C6
constrains	Governance	X limits or governs Y	$X \rightarrow Y$	P4 constrains C1
addresses	Solution	X solves or mitigates Y	$X \rightarrow Y$	FW2 addresses GB1
limits	Boundary	X restricts the scope of Y	$X \rightarrow Y$	B5 limits LA2
uses	Dependency	X operationally depends on Y	$X \rightarrow Y$	DC1 uses CA1, CA2, CA4
feeds	Lifecycle	X's output becomes Y's input	$X \rightarrow Y$	LP01 feeds LP02
gates	Lifecycle	X must pass before Y can proceed	$X \rightarrow Y$	LP08 gates LP09
resolves	Resolution	X eliminates the impossibility claim of Y	$X \rightarrow Y$	TI1 resolves VDR-2 transcendental claim

Relationship	Domain	Meaning	Directionality	Example
alternative to	Choice	X and Y serve the same purpose	$X \leftrightarrow Y$	FB7 alternative to FB6
instance of	Classification	X is a concrete example of Y	$X \rightarrow Y$	FR3 instance of FR1
component of	Containment	X is a part within Y	$X \rightarrow Y$	C12 component of C13

C.2 Relationship Type Usage Frequency Across VDR-1 through VDR-10

Relationship Type	VDR-1	VDR-2	VDR-3	VDR-4	VDR-5	VDR-6	VDR-7	VDR-8	VDR-9	VDR-10	Total
enables	4	2	2	3	2	1	0	2	1	1	18
requires	3	0	1	0	2	1	0	1	0	0	8
implements	5	0	1	3	2	0	0	1	0	1	13
uses	2	3	3	4	1	3	2	5	8	4	35
demonstrates	0	5	3	2	0	0	0	0	0	0	10
composes of	2	0	0	1	0	0	0	0	0	0	3
specialization of	3	0	0	0	0	0	0	0	0	0	3
distinct from	2	0	0	0	1	0	0	0	0	0	3
constrains	1	1	0	0	1	1	0	1	0	0	5
limits	2	3	1	0	0	0	0	0	0	0	6
feeds	0	0	0	0	0	0	6	0	0	0	6
gates	0	0	0	0	0	0	1	0	0	0	1
resolves	0	0	3	0	0	0	0	0	0	0	3
addresses	0	1	2	0	0	1	0	0	0	0	4
grounds	2	0	0	0	0	0	0	0	0	0	2

C.3 Relationship Validation Rules

Rule	Condition	Violation Type	Detection Method
Referential integrity	Both <code>from</code> and <code>to</code> IDs exist in the document	Error	ID lookup in all <code>ids()</code>
Type declared	<code>rel</code> value exists in decode legend	Warning	Enum membership check
No self-reference	<code>rel types enum</code>	Warning	String comparison
Directional consistency	<code>from</code> $\neq$ <code>to</code>	Warning	Bidirectional scan
Cross-table reference valid	If A enables B, there should not also be B enables A	OK	ID prefix determines table
ID list expansion	<code>from</code> and <code>to</code> may reference different tables	OK	Split and validate each
	<code>to</code> may be comma-separated ID list (e.g., "C2,C3,C4")		

Appendix D: Compacted Document Statistics Across VDR Series

D.1 Compaction Metrics Per Paper

Paper	Source Character	Source Words (est.)	Compacted Tokens (est.)	Tables Used	Relationships	Unique IDs	Compression %
VDR-1	Philosophy/Spec	15,000	2,800	12	35	78	81%
VDR-2	Research	12,000	2,200	8	22	54	82%
VDR-3	Research	8,000	1,600	9	18	42	80%
VDR-4	Specification	7,000	1,400	8	20	38	80%
VDR-5	Specification	18,000	3,200	10	28	72	82%
VDR-6	Specification	22,000	3,800	9	24	65	83%

Paper	Source Character	Source Words (est.)	Compacted Tokens (est.)	Tables Used	Relationships	Unique IDs	Compression %
VDR-7	Methodology	16,000	2,600	11	30	58	84%
VDR-8	Specification	14,000	2,400	10	26	52	83%
VDR-9	Specification	18,000	2,800	8	22	48	84%
VDR-10	Specification	20,000	3,400	11	32	68	83%
Total	Mixed	~150,000	~26,200	—	257	575	~83% avg

D.2 Information Density Comparison

Metric	Raw Source	Compacted	Ratio	What This Means
Words per named concept	~190	~25	7.6 ×	Compacted form defines concepts 7.6 × more densely
Words per relationship	~580	~3	193 ×	Relationships are almost pure structure; prose is pure waste
Words per claim	~450	~20	22.5 ×	Claims compress dramatically because evidence IDs replace prose
Words per boundary	~300	~15	20 ×	Limitations are concise when stripped of hedging
Unique IDs per 1000 tokens	~3.8	~22	5.8 ×	Compacted form has 5.8 × more addressable items per token

D.3 Token Type Distribution in Compacted Documents

Token Category	Percentage	Examples	Grammar-Providable?
Structural delimiters	18%	, #, (,), newlines	Yes — 100%
ID tokens	12%	P1, C3, CL7, OP12	Yes — prefix + counter

Token Category	Percentage	Examples	Grammar-Providable?
Column headers	8%	id, name, category, definition	Yes — from schema
Relationship types	5%	enables, requires, uses	Yes — from enum
Category values	4%	core, axiom, demonstrated	Yes — from enum
Table names	3%	concepts, principles, claims	Yes — from profile
Content identifiers	15%	VDR Triple, softmax, KB ASSERT	Partially — from KB vocabulary
Content text	35%	Definitions, descriptions, rationale	No — LLM judgment required

Grammar-directed generation can provide approximately 50% of compacted output tokens (structural + IDs + headers + enums + table names) for free, leaving only content identifiers (15%, partially KB-assisted) and content text (35%, LLM-required).

Appendix E: Grammar Matching Score Computation

E.1 Scoring Components

Component	Weight	Computation	Range
Requirement satisfaction	Gate (must pass)	All declared requirements met against data	Pass/fail
Slot coverage	40%	Filled slots / total slots	0/100 to 100/100
Type precision	30%	Exact type matches / total slot fills	0/100 to 100/100
Connection pattern match	20%	Pattern matches KB topology	0/100 or 100/100
Best-when relevance	10%	LLM assessment of situational fit	0/100 to 100/100

Final score = (slot_coverage $\times$ 40 + type_precision $\times$ 30 + connection_match $\times$ 20 + relevance $\times$ 10) / 100, but only if all requirements pass. Any failed requirement produces score 0.

E.2 Scoring Examples

Data Attributes	Req Grammar Met?	Coverage	Type Prec.	Conn Match	Relevance	Score
4 items, 5 numeric attrs	comparison table Yes	80% (4/5 slots)	100%	N/A (100%)	90%	85/100
4 items, 5 numeric attrs	ranked list Yes	60% (3/5 mapped)	100%	N/A (100%)	70%	71/100
4 items, 5 numeric attrs	single recommendation No (no clear dominant)	—	—	—	—	0/100
1 item, 3 attrs	comparison table No (min 2 items)	—	—	—	—	0/100
1 item, 3 attrs	detail view Yes	100%	100%	N/A (100%)	95%	97/100
3 items, tradeoff attrs	pros cons pairs Yes	60% (3/5)	100%	N/A (100%)	85%	73/100
KB with 3 sourced from connections	provenance chain Yes	100%	100%	100%	90%	97/100
KB with 0 connections	provenance chain No (requires sourced from)	—	—	—	—	0/100

E.3 Tiebreaking Rules

Priority	Rule	Rationale
1	Higher score wins	Primary selection criterion
2	More specific grammar wins	Grammar with more requirements is more targeted
3	Local grammar wins over inherited	KB-local grammar was designed for this KB's data

Priority	Rule	Rationale
4	Most recently created wins	Newer grammars reflect current preferences
5	Alphabetical by name	Deterministic tiebreaker of last resort

Appendix F: Auto-Generated Grammar Catalog

F.1 Grammars Generated Per Table Schema

Table Schema	extract grammar	detail grammar	resolve grammars	Total
principles	extract principles	detail principles	—	2
concepts	extract concepts	detail concepts	—	2
claims	extract claims	detail claims	resolve claims evidence (id list)	3
operations	extract operations	detail operations	—	2
boundaries	extract boundaries	detail boundaries	—	2
rules	extract rules	detail rules	—	2
distinctions	extract distinctions	detail distinctions	—	2
axes	extract axes	detail axes	—	2
components	extract components	detail components	—	2
builtins	extract builtins	detail builtins	—	2
constraints	extract constraints	detail constraints	—	2
entities	extract entities	detail entities	—	2
fields	extract fields	detail fields	resolve fields entity (id ref)	3
phases	extract phases	detail phases	—	2
test results	extract test results	detail test results	—	2
failures	extract failures	detail failures	—	2
findings	extract findings	detail findings	resolve findings evidence (id list)	3
benchmarks	extract benchmarks	detail benchmarks	—	2

Plus 3 document-level grammars: compact_display, document_summary, relationship_display.

Maximum grammars auto-generated per compacted KB: (tables used $\times$ 2) + resolve grammars + 3 document-level. For a philosophy compaction using 7 tables: $14 + 0 + 3 = 17$ grammars. For a specification using 10 tables with id_ref columns: $20 + 2 + 3 = 25$ grammars.

F.2 Usage Grammar Slot Specifications

Usage Type	Slots	Slot Types	Connection Direction	Template Pattern
reference	item id, item name, item detail, source ref	id ref, identifier, text, text	outbound from target to source	Inline: “{name} ({id} from {source}): {detail}”
comparison	local items, source items, com- parison columns	list of dict, list of dict, list of identifier	outbound from target to source	Merged table with source column
evidence	claim id, sup- porting ids, sup- porting data, confi- dence	id ref, id list, list of dict, fraction	inbound from source to target	Bulleted evidence list with confidence
dependency	local id, de- pends on ids, depen- dency chain	id ref, id list, list of text	outbound from target to source	Arrow chain: “A $\rightarrow$ B $\rightarrow$ C”

Usage Type	Slots	Slot Types	Connection Direction	Template Pattern
summary	title, table counts, key items, key re- lation- ships	text, dict, list of dict, list of text	outbound from target to source	Title + counts + key items

F.3 Grammar Inheritance Resolution Examples

Query Origin	Available Grammars (bottom to top)	Selected	Reason
root.project.vdr.training	training: step log format; vdr: gym result; project: code block; root: basic list	step log format	Most local match
root.project.vdr	vdr: gym result; project: code block; root: basic list	gym result	Most local
root.project.other	project: code block; root: basic list	code block	Inherited from project
root.stories.londonstories	dialogue format; root: basic list	dialogue format	Story- specific format
root.project.vdr (override)	vdr: basic list(numbered); project: code block; root: basic list(bulleted)	basic list(numbered)	Local override shadows root

Appendix G: Structural Token Analysis by Output Type

G.1 Python Code Generation Breakdown

Token Category	Example Tokens	% of Output	Grammar-Provided?	Forward Passes Saved
Keywords	def, return, for, in, if, else, class, import	12%	Yes	12%
Punctuation	(,), :, ,, ., =, [,], {, }	15%	Yes	15%

Token Category	Example Tokens	% of Output	Grammar-Provided?	Forward Passes Saved
Whitespace/indentation	spaces, newlines, indentation levels	10%	Yes	10%
Built-in names	print, len, range, True, False, None	3%	Yes (from KB)	3%
Type hints	List, Dict, Optional, int, str, bool	4%	Yes (from KB)	4%
User identifiers	function names, variable names	18%	Partially (from KB scope)	~9%
String literals	“error message”, ‘value’	8%	No	0%
Numeric literals	42, 3.14, 0	5%	Partially (from KB data)	~3%
Logic/algorithm	actual computation expressions	25%	No	0%
Total		100%	~56% fully, ~12% partially	~56%

G.2 JSON Output Breakdown

Token Category	Example Tokens	% of Output	Grammar-Provided?	Forward Passes Saved
Structure	{, }, [,], :, ,	22%	Yes	22%
Quotes	” around keys and string values	18%	Yes	18%
Key names	“name”, “value”, “status”	15%	Yes (from schema)	15%
Whitespace	indentation, newlines	8%	Yes	8%
String values	actual content strings	25%	No (LLM or KB)	~12% (KB portion)

Token Category	Example Tokens	% of Output	Grammar-Provided?	Forward Passes Saved
Numeric values	42, 3.14	7%	Partially (KB data)	~5%
Boolean/null	true, false, null	5%	Yes	5%
Total		100%	~68% fully, ~17% partially	~85%

G.3 English Prose Breakdown

Token Category	Example Tokens	% of Output	Grammar-Provided?	Forward Passes Saved
Articles	the, a, an	7%	Yes (grammar-determined)	7%
Prepositions	of, in, to, for, with, from	8%	Partially	~4%
Conjunctions	and, but, or, because	4%	Partially	~2%
Punctuation	., ,, ;, :, ?, !, —, (,)	6%	Yes	6%
Pronouns	it, they, this, that, which	5%	Partially	~2%
Common verbs	is, are, was, has, have, can	6%	Partially	~3%
Technical terms	VDR, fraction, softmax, KB	10%	Yes (from KB vocabulary)	10%
Content nouns	specific nouns for the topic	20%	No	0%
Content verbs	specific action/state verbs	12%	No	0%
Adjectives/adverbs	specific descriptors	10%	No	0%
Data values	numbers, names, references	12%	Yes (from KB)	12%
Total		100%	~25% fully, ~11% partially	~46%

G.4 Compacted Table Output Breakdown

Token Category	Example Tokens	% of Output	Grammar-Provided?	Forward Passes Saved
Hash/comment markers	#	2%	Yes	2%
Table names	concepts, principles	3%	Yes (from profile)	3%
Parentheses	() around column headers	2%	Yes	2%
Pipe delimiters	between columns	18%	Yes	18%
Column headers	id, name, category, definition	6%	Yes (from schema)	6%
Newlines	between rows	8%	Yes	8%
ID tokens	P1, C3, CL7	10%	Yes (prefix + counter)	10%
Category values	core, axiom, demonstrated	4%	Yes (from enum)	4%
Relationship types	enables, requires	3%	Yes (from enum)	3%
ID references	C2,C3,C4	5%	Partially (IDs from KB)	~3%
Content text	definitions, descriptions	39%	No (LLM judgment)	0%
Total		100%	~56% fully, ~5% partially	~59%

Appendix H: Test Suite Detailed Breakdown

H.1 Test Coverage by Component

Component	Unit Tests	Edge Case Tests	Integration Tests	Roundtrip Tests	Total
Enums	5	0	0	0	5
ColumnDef	2	0	0	0	2
TableSchema	2	0	0	0	2
STANDARD	7	0	0	0	17
TABLES					
CompactionProfile	12	0	0	0	12

Component	Unit Tests	Edge Case Tests	Integration Tests	Roundtrip Tests	Total
DecodeLegend	1	1	0	0	4
GrammarRule	4	0	0	0	4
CompactedRow	0	0	2	0	2
CompactedTable	0	0	1	0	3
CompactedDocument	1	0	2	0	6
PathRegistry	8	2	2	1	13
KnowledgeBase.facts	2	2	0	0	7
KnowledgeBase.retract	1	1	0	0	2
KnowledgeBase.query	1	1	0	0	3
Counter	4	2	0	0	6
LockState	5	2	0	0	7
reset live	4	0	0	0	4
compacted	7	0	5	0	12
doc to kb					
Grammar	7	0	0	0	7
auto-generation					
kb to	0	0	0	11	11
com-pacted					
doc					
generate	10	0	2	0	12
usage					
grammar					
CompactionPipeline.classify	0	0	0	0	7
CompactionPipeline.applicable	0	0	0	0	2
CompactionPipeline.compact			12	0	12
Cross-KB	0	0	7	0	7
integration					
Grammar	0	0	6	0	6
inheritance					
Total	112	12	39	12	179

H.2 The One Failing Test

Test Name	Expected	Actual	Root Cause	Fix
doc.all ids has 5 IDs	{P1, P2, C1, C2, C3}	Superset including empty strings	all ids() collects from all tables including relationships and section index whose rows have id=" "	Filter: re- turn [id for id in ids if id]

Classification: test expectation calibration, not system bug. Consistent with VDR series pattern (13 test-authoring errors across 10 papers, zero computation errors).

H.3 Untested Areas (Planned for Future)

Area	Why Untested	Priority	Planned Stage
Grammar matching score computation	Requires Prolog engine integration	High	Stage 3
Slot filling from connected KBs	Requires mount system	Medium	Stage 4
Grammar evolution (modify/retire)	Requires usage tracking	Medium	Stage 4
Concurrent grammar access	Requires session cloning	Low	Stage 5
Large document compaction (>50K words)	Requires performance testing	Medium	Stage 4

Area	Why Untested	Priority	Planned Stage
Grammar conflict detection (two grammars claim same situation)	Requires tiebreaking rules	Medium	Stage 3
Decode legend constraint enforcement	Requires Prolog condition evaluation	High	Stage 2
Cross-document relationship discovery	Requires graph traversal builtins	Medium	Stage 4

Appendix I: Compaction vs Summarization Comparison

I.1 What Each Preserves and Loses

Aspect	Compaction	Summarization
Named concepts	ALL preserved with exact names	Selected subset; names may be paraphrased
Definitions	ALL preserved (compressed to one line)	Key definitions only; may be shortened further
Relationships	ALL typed edges preserved	Only “most important” relationships mentioned
Constraints	ALL preserved with scope and violation	Rarely mentioned
Boundaries/limitations	ALL preserved	May be omitted as “less important”
Claims with evidence refs	ALL preserved with evidence IDs	Key claims; evidence links lost
Quantitative data	ALL exact values preserved	Rounded or omitted
Section provenance	Complete section→ID mapping	Lost entirely
Type system (decode legend)	Complete enum declarations	Lost entirely

Aspect	Compaction	Summarization
Prose flow	Lost entirely	Partially preserved (summary reads like text)
Explanatory examples	Lost (unless they demonstrate something the rule alone doesn't)	Selected examples preserved
Worked demonstrations	Lost (unless load-bearing)	Summarized or omitted
Author voice and emphasis	Lost	Partially preserved

I.2 Information Recovery Comparison

Query Type	From Compacted Data	From Summary
“What is concept X?”	Exact: query row by ID → definition	Depends: may or may not be in summary
“What does X depend on?”	Exact: query relationships where from=X	Unknown: relationship likely not in summary
“What are the limitations?”	Exact: query boundaries table → all limitations	Partial: maybe 1-2 mentioned
“How many tests passed?”	Exact: query test results table → exact counts	Approximate: “hundreds of tests”
“What constraint governs X?”	Exact: query constraints table	Lost: constraints rarely survive summarization
“Trace claim CL3 to its evidence”	Exact: CL3 → evidence ID list → evidence rows	Impossible: evidence chain not preserved
“What was defined in section 5?”	Exact: section index → IDs → rows	Impossible: section mapping not preserved
“List all operations”	Exact: query operations table → complete list	Partial: selected operations mentioned

I.3 Size Comparison for VDR-1

Representation	Token Count	Information Completeness	Queryable?
Full paper	~15,000	100% (by definition)	No (unstructured text)
Compacted tables	~2,800	~95% structural, 0% prose	Yes (typed columns, ID lookup)

Representation	Token Count	Information Completeness	Queryable?
GPT-style summary	~1,500	~40% (key points only)	No (unstructured text)
Abstract only	~300	~10% (overview only)	No (unstructured text)

The compacted form is $5.4 \times$ smaller than the full paper while preserving 95% of structural information. The summary is $1.9 \times$ smaller than the compacted form but preserves only 40% of information and is not queryable.

Appendix J: Connection-Aware Grammar Patterns

J.1 Connection Topology Signatures

Topology Pattern	Meaning	Matching Grammars	Example KB
No connections	Standalone KB	detail, summary, compact display	Freshly created notebook
1+ inbound sourced from	Has provenance	provenance chain, source attribution	Corpus KB
1+ outbound evaluated by	Has been evaluated	eval dashboard, quality report	Model checkpoint
inbound sourced from + outbound evaluated by	Trained and evaluated	model status dashboard	Deployment candidate
1+ inbound evidence source	Used as evidence	evidence summary, citation list	Referenced paper KB
1+ outbound depends on	Has dependencies	dependency tree, dependency table	Inference notebook
1+ outbound + 1+ inbound (same rel)	Bidirectional connection	collaboration view	Shared editing KB
0 outbound, many inbound	Terminal/leaf node	detail view, archive summary	Retired model
Many outbound, 0 inbound	Root/source node	overview dashboard	Source registry

J.2 Multi-Connection Grammar Slot Filling

Grammar	Slot	Filled From	Connection Followed	Access Cost
model status dash- board.training loss	fraction	Training run KB	outbound trained by	1 hop, O(1) by integer ID
model status dash- board.eval scores	table	Eval result KBs	outbound evaluated by	1 hop per eval, O(n)
model status dash- board.safety rate	fraction	Safety eval KB	outbound evaluated by (filtered by eval type)	1 hop + 1 filter
model status dash- board.deployment status	categorical	Deployment KB	outbound deployed as	1 hop
provenance chain.source names	list	Source KBs	inbound sourced from (recursive)	Walk chain to leaves
evidence sum- mary.supporting facts	list of dict	Evidence source KBs	inbound evidence source	1 hop per source
dependency tree.full chain	list of text	Dependency target KBs	outbound depends on (transitive)	Walk chain

J.3 Connection Pattern Syntax

Pattern	Meaning	Prolog Evaluation
has inbound (REL, 1+)	At least 1 inbound connection of type REL	member (C, KB.connections), C.direction=inbound, C.relationship=REL
has out- bound (REL, 1+)	At least 1 outbound connection of type REL	Same with direction=outbound
has inbound (REL, N)	Exactly N inbound of type REL	Count matches = N
has inbound (REL, 0)	No inbound of type REL	No matches found

Pattern	Meaning	Prolog Evaluation
has any inbound	At least 1 inbound of any type	Any inbound connection exists
has inbound (REL1, 1+) and has out-bound (REL2, 1+)	Both conditions	Both patterns satisfied
not (has inbound (REL,))	No inbound of type REL	Pattern match fails

Appendix K: Compaction Pipeline Decision Points

K.1 Where Prolog Decides vs Where LLM Decides

Decision	Decider	Input	Output	Confidence
Source character (clear signals)	Prolog	Keyword counts	SourceCharacter1/1 enum (deterministic)	
Source character (ambiguous)	LLM	Full source text	SourceCharacter70-90/100 enum	
Profile selection	Prolog	SourceCharacter	CompactionProfile1/1 (lookup)	
Required tables	Prolog	Profile	Table list	1/1 (declared)
Optional table applicable?	Prolog	Schema applicable when + source text	Boolean	90/100 (keyword heuristic)
ID prefix assignment	Prolog	Profile.id prefix scheme	Prefix map	1/1 (declared)
What constitutes a “concept”	LLM	Source text	Concept rows	70-85/100
What constitutes a “principle”	LLM	Source text	Principle rows	75-90/100
What constitutes a “relationship”	LLM	Source text + extracted IDs	Relationship edges	65-80/100
How to compress a definition to one line	LLM	Original prose + concept name	Compressed text	70-85/100

Decision	Decider	Input	Output	Confidence
Section-to-ID mapping	Hybrid	Source sections + extracted IDs	Section index rows	85-95/100
Decode legend enum collection	Prolog	Schemas used	Legend entries	1/1 (deterministic)
Constraint generation from legend	Prolog	Legend entries	Constraints	1/1 (deterministic)
Grammar generation from schemas	Prolog	Table schemas	Grammar rules	1/1 (deterministic)
Validation against enums	Prolog	Values + enum declarations	Pass/fail per value	1/1 (deterministic)

K.2 LLM Judgment Quality Factors

Factor	Improves Quality	Degrades Quality
Source clarity	Well-structured source with clear headings and definitions	Dense prose with implicit concepts
Domain familiarity	Source domain well-represented in training data	Novel domain with specialized terminology
Compaction examples	LLM has seen compacted examples of similar documents	First compaction of a new document type
KB context	Relevant KBs in scope provide terminology and relationship patterns	No related KBs in scope
Source length	Shorter documents (less ambiguity about what's important)	Very long documents (judgment fatigue)
Explicit structure	Source has numbered items, tables, lists	Source is continuous prose

Appendix L: Implementation File Structure Reference

L.1 Current Implementation (Single File)

Section	Lines (approx.)	Contains
Imports and header	1-10	stdlib imports
Enums (7)	11-80	SourceCharacter, ColumnType, Direction, MountMode, Visibility, ConstraintScope, ConstraintStatus
Simple dataclasses	81-250	ColumnDef, Connection, Constraint, Fact, Counter, LockState, DecodeLegendEntry, DecodeLegend, GrammarRule
Schema dataclasses	251-350	TableSchema, CompactionProfile
Document dataclasses	351-500	CompactedRow, CompactedTable, CompactedDocument
KnowledgeBase	501-650	Full KB struct with as- sert/retract/query/reset
PathRegistry	651-750	Path↔ID mapping
STANDARD	751-1050	17 table schema registrations
TABLES		
PROFILES	1051-1200	6 compaction profile definitions
compacted doc to kb	1201-1350	Document→KB conversion + grammar generation
kb to compacted doc	1351-1450	KB→Document reconstruction
generate usage grammar	1451-1600	5 usage type grammar generators
CompactionPipeline	1601-1750	Full pipeline with classification

L.2 Planned Multi-File Structure (Stage 3+)

File	Extracted From	Dependencies
core/types.py	Enums + simple dataclasses	None
core/kb.py	KnowledgeBase + Counter + LockState + Fact + Constraint	core/types.py
core/path registry.py	PathRegistry	None
core/prolog.py	(new — Prolog engine)	core/types.py, core/kb.py
compaction/schemas.py	STANDARD TABLES + TableSchema + ColumnDef	core/types.py
compaction/profiles.py	PROFILES + CompactionProfile	compaction/schemas.py
compaction/document.py	CompactedRow + CompactedTable + CompactedDocument	core/types.py
compaction/grammar.py	GrammarRule + auto-generation + usage grammar	compaction/schemas.py, core/kb.py
compaction/pipeline.py	CompactionPipeline	All compaction modules
compaction/legend.py	DecodeLegend + DecodeLegendEntry	core/types.py

END VDR-12 APPENDIX TABLES

[[HOWL-VDR-12-2026](#)] Grammar-Directed Compaction and Generation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-12-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-3-2026](#)] VDR Gym Extension. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-3-2026>

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026>

[[HOWL-LLM-1-2026](#)] Integer LLM with Prolog Knowledge Base. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026>

[[HOWL-VDR-5-2026](#)] Exact Arithmetic Meets Logical Provenance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026>

[[HOWL-VDR-6-2026](#)] Computational Primitives and Operational Environments. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-6-2026>

[[HOWL-VDR-7-2026](#)] Complete Lifecycle Technical Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-7-2026>

[[HOWL-VDR-8-2026](#)] Computational State Primitives, Universal Data Pathing, and Session Management. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026>

[[HOWL-VDR-9-2026](#)] Orchestrated Inference. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-9-2026>

[[HOWL-VDR-10-2026](#)] Operational Foundations and Comprehensive Builtin Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-10-2026>

[[HOWL-VDR-11-2026](#)] Implementation Blueprint. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-11-2026>